Качество данных на примере HMDA

Мы проверим знакомую таблицу ипотечных записей, затем её учебную копию с дефектами. Итог — порядок очистки с сохранением исходных значений, причинами исключений и проверкой аналитического результата. Поля и данные показаны до применения методов.

Слайд 1. Качество данных

На прошлом занятии мы считали долю одобрений. Теперь представим, что нам прислали ещё одну выгрузку тех же записей. Перед расчётом нужно разобраться, что в ней находится: где значение действительно неизвестно, где число записано текстом и какие строки пришли повторно.

К концу пары у нас будет понятный порядок работы: осмотреть файл, записать правила проверки, найти нарушения и сохранить результат вместе с причинами исключений. На практике вы выполните этот порядок сами. Начнём со знакомой таблицы и вспомним, что означает её строка.

Слайд 2. Знакомая таблица HMDA

Перед нами первые четыре записи decisions.csv. Одна строка — опубликованная ипотечная запись. В столбце action_taken находится исход: 1 — кредит выдан, 2 — заявка одобрена, но предложение не принято, 3 — отказ. У показанных четырёх строк исход 1; в полном файле встречаются все три кода.

source_row помогает найти строку в сохранённом исходном снимке. Например, у первой показанной записи номер 2. loan_amount — сумма кредита в долларах, income — годовой доход в тысячах долларов: значение 72 означает 72 тысячи долларов в год. Пустая клетка означала бы, что доход неизвестен. Сейчас загрузим файл и увидим те же поля в выводе Python.

source_row	action_taken	loan_amount	income
2	1	275000	72
3	1	585000	174
5	1	275000	250
7	1	295000	63

```
decisions.head(4)
```

Слайд 3. Словарь полей

На предыдущем слайде были четыре поля. В полном файле есть ещё три: activity_year, county_code и lei. activity_year — отчётный год, здесь 2023. county_code — код территории, здесь 11001, Вашингтон. Это код, поэтому сохраняем его текстом. lei — код организации, передавшей запись. Одна организация передаёт много записей, поэтому её код повторяется.

Важно различать lei и source_row. По lei можно собрать все записи организации. По source_row в этом сохранённом снимке можно найти конкретную исходную строку. Это технический номер нашей копии, а не номер клиента. Выведем список столбцов и типы. Тип float у суммы означает способ хранения числа; единицу измерения мы узнаём из словаря, который сейчас перед нами.

Поле	Значение
source_row	Номер строки исходного снимка
activity_year	Отчётный год
lei	Код организации
county_code	Код территории
action_taken	Исход: 1 — выдача, 2 — одобрение, 3 — отказ
loan_amount	Сумма кредита, USD
income	Доход за год, тыс. USD

```
print(decisions.columns.tolist())
print(decisions.dtypes)
```

Слайд 4. Правила для нашего расчёта

Вспомним расчёт: считаем записи с кодом 1 или 2 и делим на число всех записей файла. В оригинальном decisions.csv их 6830; одобрений 6139. Получаем 89,88 процента. Это контроль для сравнения, когда начнём менять состав строк.

Запишем, какой вход ожидаем: известные коды решений, числовая положительная сумма, код территории 11001. Для технического номера проверим повторы. Такое описание называют контрактом данных: у каждого поля есть правило и действие при нарушении. Сумму и территорию проверяем при подготовке всей таблицы, хотя в формуле доли используется только исход. Доход может отсутствовать: мы можем сосчитать одобрения и без него. Если будем считать средний доход, вопрос о пропусках решим отдельно.

Поле	Проверка	Действие
source_row	Известен, без повторной доставки	Разобрать повторы
action_taken	Код 1, 2 или 3	Отложить нарушение
loan_amount	Число больше нуля	Отложить нарушение
county_code	Код 11001	Уточнить территорию
income	Может отсутствовать	Сохранить и посчитать пропуски

```
approved = decisions["action_taken"].isin([1, 2]).sum()
n = len(decisions)
print(approved, n, f"{100 * approved / n:.2f}%")
```

Слайд 5. Профиль пропусков

Начинаем с info: он показывает названия столбцов, типы и количество известных значений. Затем isna для каждой клетки возвращает логическое значение: True при пропуске, False при известном значении. sum по столбцам считает True. Получается краткая сводка пропусков.

В income обнаружим 342 пропуска. В action_taken пропусков нет. Выберем несколько строк с неизвестным доходом и покажем рядом их исходы. loc получает маску строк и список нужных столбцов; head оставляет первые пять для просмотра. Мы видим, что неизвестный доход и известный исход могут находиться в одной записи. Поэтому пока просто регистрируем проблему, не удаляя строку.

```
decisions.info()
```

```
print(decisions.isna().sum())
print(decisions["action_taken"].value_counts(dropna=False))
unknown_income = decisions["income"].isna()
decisions.loc[unknown_income, ["source_row", "action_taken", "income"]].head(5)
```

Слайд 6. Пропуск и ноль

Возьмём отдельный пример, не изменяя HMDA: доходы 100, 200 и неизвестное значение. pd.Series создаёт одномерный ряд, а float("nan") обозначает отсутствующее число. Метод mean по умолчанию пропускает неизвестные значения: складывает 100 и 200 и делит на два. Получается 150.

fillna(0) возвращает новый ряд, в котором вместо пропуска стоит ноль. Теперь mean складывает три известных числа и делит на три. Получается 100. Мы не узнали доход третьего человека, а добавили своё предположение. В исходном файле оставим неизвестное значение. Когда понадобится средний доход, укажем, по скольким известным значениям он посчитан.

```
illustration = pd.Series([100.0, 200.0, float("nan")])
print(illustration)
print(illustration.mean())
print(illustration.fillna(0).mean())
```

Слайд 7. Доля пропусков по исходам

Число пропусков само по себе зависит от размера группы. У выдач их 244, у одобрений без принятия предложения — 33. Но выдач гораздо больше. Для сравнения разделим пропуски на количество записей своего исхода.

Сначала создаём копию таблицы и столбец income_missing из True и False. groupby разбивает её по action_taken. В agg задаём два результата: size считает все строки группы, sum считает True в income_missing. reset_index возвращает код исхода в обычный столбец. После этого отдельным действием делим missing на n. Получим примерно 4,10; 18,03 и 9,41 процента. Такая таблица показывает различие частот. Причину отсутствия дохода из неё пока не узнаём.

action_taken	Все записи	Нет income	Доля
1	5956	244	4,10%
2	183	33	18,03%
3	691	65	9,41%

```
missing_input = decisions.copy()
missing_input["income_missing"] = missing_input["income"].isna()
missing_profile = missing_input.groupby("action_taken").agg(
    n=("income", "size"),
    missing=("income_missing", "sum"),
).reset_index()
missing_profile["missing_rate"] = missing_profile["missing"] / missing_profile["n"]
missing_profile
```

Слайд 8. Удаление строк с неизвестным доходом

dropna с subset проверяет пропуски только в перечисленных полях. Здесь он возвращает таблицу, где известен income. В decisions остаются все 6830 записей, а в complete_income — 6488. Сам исходный DataFrame не меняется, потому что результат записан под другим именем.

Выполним одну и ту же формулу для обеих таблиц. Среди всех решений было 89,88 процента одобрений, среди записей с известным доходом получается 90,35. Доля выросла без изменения самих исходов: мы исключили часть записей, причём исходы

потеряли разную долю строк. Для исходного вопроса используем все известные решения. Для вопроса о доходах отдельно называем группу с известным income и её размер.

```
complete_income = decisions.dropna(subset=["income"])
print(len(decisions), len(complete_income))
for label, part in [("Все решения", decisions), ("Известный доход", complete_income)]:
    k = part["action_taken"].isin([1, 2]).sum()
    print(label, len(part), k, k / len(part))
```

Слайд 9. Причина отсутствия значения

Предположим, что доход не заполнили из-за сбоя выгрузки. Или его не собирали для определённого вида записи. Или отсутствие связано с самим значением дохода. Эти объяснения приводят к разным способам дальнейшего анализа, но одна пустая клетка не позволяет выбрать объяснение.

На этом этапе достаточно записать наблюдаемое: какое поле отсутствует, сколько таких строк и с какими доступными признаками это связано. В нашем файле частоты различаются по исходам. Следующий вопрос поставщику: когда income может отсутствовать и как это отражается в опубликованном наборе? До ответа мы сохраняем пропуски и явно называем группу, по которой считаем среднее. Статистическое восстановление неизвестных значений требует отдельного обоснования и здесь не выполняется.

Слайд 10. Учебная копия выгрузки

До этого мы работали с оригинальным decisions.csv. Теперь открываем training_export.csv. Для упражнения в копии изменили несколько значений и добавили две повторные доставки. Это отдельный файл; оригинал остаётся рядом для сравнения.

Покажем первые четыре строки. Там, где раньше была сумма 275000, теперь записано oops. В action_taken одной записи появился 88. В income другой — BAD. В код территории добавили внешние пробелы. Прочитаем все поля как string: тогда Python не попробует сам угадать числовой тип и мы сохраним исходные обозначения. Пустые клетки read_csv по-прежнему распознаёт как пропуски. Сначала найдём каждое нарушение, затем решим, что делать с затронутой строкой.

source_row	Поле	Было в оригинале	В учебной копии
2	loan_amount	275000.0	oops
3	action_taken	1	88
5	income	250.0	BAD
7	county_code	11001	" 11001 "

```
export = pd.read_csv(LOCAL / "training_export.csv", dtype="string")
export.head(5)
print(export.shape)
print(export.dtypes)
raw_copy = export.copy(deep=True)
```

Слайд 11. Текст в число и ошибки преобразования

pd.to_numeric получает Series с текстом и возвращает числовую Series. Обычная строка "275000.0" превращается в число. Для oops число получить нельзя. Параметр errors="coerce" выбирает реакцию: поставить отсутствующее значение вместо остановки с ошибкой.

После преобразования одного isna недостаточно: он найдёт и старые пропуски, и вновь возникшие. Поэтому берём два условия: исходное поле было известно — notna; полученное число неизвестно — isna. Объединяем их знаком &: оба условия проверяются для каждой строки. Маской выбираем исходные source_row и loan_amount. В выводе увидим oops. На практике тем же способом отдельно проверите income и найдёте BAD.

```
numeric_amount = pd.to_numeric(export["loan_amount"], errors="coerce")
parse_error = export["loan_amount"].notna() & numeric_amount.isna()
print(export.loc[parse_error, ["source_row", "loan_amount"]])
```

Слайд 12. Пробелы и коды категорий

Код территории — текст. Доступ str позволяет применить строковый метод ко всем значениям Series. strip убирает пробелы в начале и конце: " 11001 " превращается в "11001". Перед заменой сравним старые и новые значения с помощью ne — «не равно» — и выведем затронутые строки.

С кодом исхода другая ситуация. 88 успешно превращается в число, но по нашему словарю допустимы только 1, 2 и 3. Поэтому после to_numeric нужен isin. Знак ~ меняет True на False и наоборот; так получаем маску недопустимых исходов. Для 88 мы не знаем правильного исхода по содержимому экспортного файла. Сохраним запись отдельно и запросим исправление. Внешние пробелы можем убрать по известному формату кода.

```
normalized_county = export["county_code"].str.strip()
print(export.loc[export["county_code"].ne(normalized_county), ["source_row", "county_code"]])
outcome_num = pd.to_numeric(export["action_taken"], errors="coerce")
invalid_outcome = ~outcome_num.isin([1, 2, 3])
print(export.loc[invalid_outcome, ["source_row", "action_taken"]])
```

Слайд 13. Единицы измерения

Возьмём уже показанные сумму 275000 и доход 72. Сумма опубликована в долларах, доход — в тысячах долларов за год. Чтобы перевести доход в доллары за год, умножаем его на 1000. Чтобы показать сумму кредита в тысячах долларов, делим её на 1000.

Оба поля могут храниться как float, но это не делает их единицы одинаковыми. Поэтому новому столбцу даём имя с единицей: income_usd, loan_amount_thousand_usd. В коде создадим отдельную копию prepared и добавим в неё доход в долларах. Исходный income сохраним, чтобы было понятно происхождение результата. Проверка типа обнаружит текст вместо числа; ошибку в единицах найдём сравнением со словарём.

Поле	Единица	Пример
loan_amount	USD	275000
income	тыс. USD в год	72
income_usd	USD в год	72000

```
prepared = decisions.copy()
prepared["income_usd"] = prepared["income"] * 1000
prepared[["income", "income_usd"]].head(4)
```

Слайд 14. Повторы организации и строки

duplicated проверяет совпадения в Series или в выбранных столбцах DataFrame. По умолчанию первое вхождение не отмечается, последующие получают True. sum

считает такие повторные вхождения. Параметр keep=False отметит всех участников совпадающей группы, если нужно посмотреть их рядом.

Проверим lei: повторов будет много, потому что у организации много записей. Проверим source_row оригинала: повторов нет. Если исключить source_row и сравнить остальные шесть полей, найдутся 73 повторных вхождения. Покажем несколько таких записей с их разными source_row. Одинаковые опубликованные значения ещё не дают нам сведений, что это одна и та же заявка. Поэтому выбор столбцов для проверки всегда сопровождаем вопросом: какую именно повторную сущность мы ищем?

Что сравниваем	Повторяется	Наше действие
lei	Организация в разных записях	Сохраняем строки
Шесть полей без source_row	Опубликованные значения	Показываем совпадения
source_row в экспорте	Доставка исходной строки	Сравниваем всю запись

```
print("Повторы LEI:", decisions["lei"].duplicated().sum())
print("Повторы технического ключа:", decisions["source_row"].duplicated().sum())
business = decisions.drop(columns="source_row")
print(business.duplicated().sum())
assert business.duplicated().sum() == 73
pairs = business.duplicated(keep=False)
decisions.loc[pairs].head(6)
```

Слайд 15. Подтверждённые повторы доставки

В учебном экспорте повторно добавлены строки с source_row 15 и 16. Мы знаем назначение технического номера и можем проверить содержимое. Выберем все вхождения повторяющихся номеров через duplicated(keep=False), а затем посмотрим полные строки.

Перед удалением проверим конфликт. Уберём только полностью одинаковые строки временного представления: drop_duplicates без subset сравнивает все столбцы. Если после этого один известный source_row всё ещё повторяется, значит у него разные версии. assert с is_unique остановит такой случай. Только после успешной проверки оставим первое вхождение каждого номера. Размер станет 6830 вместо 6832. Исходные пропуски номера разбираем отдельно: неизвестные номера не позволяют считать две строки одной доставкой.

```
known_key = export["source_row"].notna()
repeat_mask = known_key & export["source_row"].duplicated(keep=False)
export.loc[repeat_mask]
repeated = export.loc[known_key].drop_duplicates()
assert repeated["source_row"].is_unique, "Конфликтующие версии ключа"
replay_mask = known_key & export["source_row"].duplicated()
unique_export = export.loc[~replay_mask].copy()
print(len(export), len(unique_export))
```

Слайд 16. Необычно большая сумма

Текст oops не задаёт число, а большая числовая сумма может быть реальной. Чтобы выделить необычные суммы для просмотра, воспользуемся квартилями. Q1 — значение, ниже которого примерно четверть наблюдений; Q3 — значение, ниже которого примерно три четверти. Между ними находится средняя половина. Разность Q3 и Q1 называется межквартильным размахом, IQR.

quantile([0.25, 0.75]) возвращает две границы. В нашем наборе Q1 равен 315000, Q3 — 755000 долларов. Прибавим к Q3 полтора размаха: получим 1415000. Суммы выше порога отметим для проверки. Их 355. Это способ выбрать необычные значения для

просмотра. Решение об исключении потребует выяснить, нарушают ли они смысл поля или условия анализа.

```
amount = decisions["loan_amount"]
q1, q3 = amount.quantile([.25, .75])
iqr = q3 - q1
upper_flag = q3 + 1.5 * iqr
print(q1, q3, upper_flag)
flagged = decisions.loc[amount.gt(upper_flag)]
print("Кандидатов на разбор:", len(flagged))
flagged[["source_row", "loan_amount", "action_taken"]].head(5)
```

Слайд 17. Среднее и медиана суммы

Среднее — сумма всех значений, делённая на их количество. Очень большая сумма увеличивает его. Медиана — середина отсортированного ряда; при чётном количестве берём среднее двух центральных значений. Поэтому она меньше реагирует на величину самых больших сумм.

В исходном наборе средняя сумма около 670199 долларов, медиана 505000. Теперь создадим without_tail, где суммы не выше вычисленного порога, и снова посчитаем среднее. Оно уменьшится. Важно назвать, для какой группы получено каждое число: первый результат относится ко всем нашим решениям, второй — к ограниченной по сумме группе. Оба расчёта полезны для сравнения чувствительности. Исходный набор decisions оставим целым.

```
print("Исходный mean/median:", amount.mean(), amount.median())
without_tail = decisions.loc[amount.le(upper_flag)]
print("Без флага хвоста:", len(without_tail), without_tail["loan_amount"].mean())
```

Слайд 18. Пересечение условий

Построим две уже знакомые маски на исходной таблице: неизвестный доход и отказ. Первая содержит 342 True, вторая — 691. Если сложить эти количества, получим 1033. Но 65 записей попали в обе группы и были посчитаны дважды.

Вертикальная черта | даёт True, если для строки выполнено хотя бы одно условие; & требует оба. Поэтому объединение содержит 968 строк, пересечение — 65. Равенство для проверки: 342 плюс 691 минус 65 равно 968. Здесь отказ — просто второе условие для примера, сами отказы мы сохраняем в анализе. В журнале качества по той же причине отдельно считаем частоту каждой проблемы и число уникальных строк, затронутых хотя бы одной проблемой.

```
missing_mask = decisions["income"].isna()
denial_mask = decisions["action_taken"].eq(3)
print(missing_mask.sum(), denial_mask.sum())
print((missing_mask | denial_mask).sum(), (missing_mask & denial_mask).sum())
```

Слайд 19. Карантин и баланс

В учебной копии мы умеем убрать внешние пробелы и точные повторные доставки. Для двух других строк правильное значение по экспорту восстановить нельзя: в одной сумма oops, в другой исход 88. Поместим их в отдельную таблицу quarantine. Это и называем карантином: строки остаются доступными вместе с причиной, но не входят в подготовленный слой clean.

Строку с BAD в income сохраним в clean: после преобразования у неё неизвестен доход, а исход известен. Ошибку преобразования запишем в журнал. Получаем баланс: 6832 входных строки состоят из 6828 строк clean, двух карантинных записей и двух повторных доставок. Проверим равенство, чтобы обнаружить незаметную

потерю строк. Оно подтверждает учёт количества; причины распределения мы только что задали отдельно.

Часть результата	Строк	Что произошло
Входной экспорт	6832	До исправлений
Повторные доставки	2	Оставлено первое вхождение
Карантин	2	Сумма oops или исход 88
Подготовленные записи	6828	Удовлетворяют правилам

Слайд 20. Функция очистки

Теперь соберём уже разобранные шаги в clean_export. source — входная таблица, out — её копия. Сначала проверяем разные версии одного известного номера, затем исключаем точные повторные доставки. После этого убираем пробелы и по очереди преобразуем четыре поля в числа, сохраняя число ошибок каждого преобразования.

Для каждой оставшейся строки создаём текст reason. При нарушении правила дописываем причину: отсутствует номер, неверна сумма, исход или территория. Где reason пуст, получаем clean. Для карантина берём исходные строки source: там остаются oops и 88, а рядом добавляется причина. audit — обычный словарь с количеством входных строк, повторов, clean, карантина и ошибок преобразования. return возвращает эти три объекта. На экране готовая функция; прочитаем её по этапам и выполним один вызов.

```
clean, quarantine, audit = clean_export(export)
print(audit)
quarantine
```

Слайд 21. Проверки результата и входа

assert проверяет утверждение. Если оно истинно, выполнение продолжается. Если ложно, возникает AssertionError и дальнейший код не выполняется. Проверим уникальность source_row, допустимость исходов, положительность сумм и баланс. Для логического ряда all означает: условие должно быть истинно в каждой строке.

Для сохранности входа используем assert_frame_equal из pandas.testing. Он сравнивает таблицы вместе с типами и индексами. Сравним export с raw_copy, сделанной при загрузке: исходник должен остаться тем же. Затем ещё раз вызовем функцию на том же export и сравним clean, карантин и журнал. Совпадение показывает повторяемость действий. Определения правил проверяем содержательно: например, заполнение дохода нулём тоже можно написать так, что два запуска совпадут.

```
from pandas.testing import assert_frame_equal
assert_frame_equal(export, raw_copy)
assert clean["source_row"].is_unique
assert clean["action_taken"].isin([1, 2, 3]).all()
assert clean["loan_amount"].gt(0).all()
assert len(export) == len(clean) + len(quarantine) + audit["replays"]
again, quarantine_again, audit_again = clean_export(export)
assert_frame_equal(clean, again)
assert_frame_equal(quarantine, quarantine_again)
assert audit == audit_again
```

Слайд 22. Показатель после очистки

Применим одну функцию расчёта к исходным данным и результату очистки. В каждом случае отдельно печатаем количество одобрений и размер таблицы. В оригинале 6139 одобрений среди 6830 решений. В clean осталось 6137 одобрений среди 6828 записей: две строки с дефектами обязательных полей были исходно выдачами и ушли в карантин.

После округления до двух знаков оба процента выглядят одинаково: 89,88. Однако числители и знаменатели различаются. Поэтому в отчёте нужны и количества, и причины исключений. Отдельно посчитаем ту же долю среди clean с известным income. Подпись обязательно называет эту группу. Количество строк после отбора поможет читателю понять, за счёт чего изменился ответ.

```
def show_approval(label, table):
    n_rows = len(table)
    assert n_rows > 0
    n_approved = table["action_taken"].isin([1, 2]).sum()
    print(label, n_approved, n_rows, f"{100 * n_approved / n_rows:.2f}%")

show_approval("Оригинал", decisions)
show_approval("После учебной очистки", clean)
complete_clean = clean.dropna(subset=["income"])
show_approval("Известный доход после очистки", complete_clean)
```

Слайд 23. Файлы результата и происхождение

Передаём не только clean.csv. В quarantine.csv сохраняем исходные значения спорных записей и reason. В quality_report.json — численный журнал audit. Код функции объясняет последовательность преобразований. Отдельный README говорит, какой вопрос решали и почему выбрали эти правила.

Такая связь между исходным файлом, преобразованием и результатом называется происхождением данных, или data lineage. В карточке снимка уже сохранены источник, параметры и контрольные суммы файлов. Контрольная сумма помогает проверить, что байты исходника не изменились. В README дополнительно укажем, что работали с учебной копией, какие исправления делали, сколько строк исключили и какой знаменатель использован. Читатель сможет повторить расчёт и найти исходную спорную запись по source_row.

Файл	Содержание
clean.csv	Подготовленные записи
quarantine.csv	Исходные спорные строки и reason
quality_report.json	Количество строк и ошибок
README.md	Вопрос, правила, результаты, ограничения

Слайд 24. Практика очистки

Проверим, что порядок понятен. Что делаем с неизвестным income при подсчёте одобрений? Сохраняем строку. Что делаем с текстом oops в обязательной сумме? Записываем ошибку и сохраняем исходную строку в карантин. Что делаем с одинаковым source_row? Сначала сравниваем полные записи; одинаковую повторную доставку исключаем, разные версии требуют разбора.

После перерыва откройте методичку P03 и ноутбук практики. В начале уже есть загрузка исходного и учебного файлов. Вы осмотрите данные, самостоятельно закончите проверку дохода и выделение карантина, затем проверите функцию и

сохраните результат. Условия находятся в методичке, письменные ответы — в отдельном отчёте. Ноутбук содержит заголовки и код. Домашняя работа запускается в отдельном ноутбуке и проверяет перенос этих действий на новые условия.

Документация и данные

Карточка исходных данных (../../../data/card.md). Описание учебной копии (../../../practices/P03/data/card.md).

Методы pandas 2.2: isna (https://pandas.pydata.org/pandas-docs/version/2.2/reference/api/pandas.DataFrame.isna.html), to_numeric (https://pandas.pydata.org/pandas-docs/version/2.2/reference/api/pandas.to_numeric.html), drop_duplicates (https://pandas.pydata.org/pandas-docs/version/2.2/reference/api/pandas.DataFrame.drop_duplicates.html), quantile (https://pandas.pydata.org/pandas-docs/version/2.2/reference/api/pandas.Series.quantile.html), assert_frame_equal (https://pandas.pydata.org/pandas-docs/version/2.2/reference/api/pandas.testing.assert_frame_equal.html).